



Threagile

Agile Threat Modeling

Threat Model Report

VaultNote Threat Model

18 April 2026

VaultNote Security Team

Table of Contents

Results Overview

Management Summary	4
Impact Analysis of 5 Initial Risks in 3 Categories	5
Risk Mitigation	6
Asset Register	7
Impact Analysis of 5 Remaining Risks in 3 Categories	10
Application Overview	11
Data-Flow Diagram	12
Security Requirements	13
Abuse Cases	14
Tag Listing	15
STRIDE Classification of Identified Risks	17
Assignment by Function	19
RAA Analysis	20
Data Mapping	21
Out-of-Scope Assets: 0 Assets	22
Potential Model Failures: 0 / 0 Risks	23
Questions: 0 / 0 Questions	24

Risks by Vulnerability category

Identified Risks by Vulnerability category	25
Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 / 2 Risks	26
Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk	29
Stage II - Custom-Developed Asset With No Build Pipeline Security: 2 / 2 Risks	31

Risks by Technical Asset

Identified Risks by Technical Asset	33
API Server: 2 / 2 Risks	34
Browser SPA: 2 / 2 Risks	38
Nginx Reverse Proxy: 1 / 1 Risk	40
MinIO Object Storage: 0 / 0 Risks	43
PostgreSQL Database: 0 / 0 Risks	45
Redis Cache: 0 / 0 Risks	47

Data Breach Probabilities by Data Asset

Identified Data Breach Probabilities by Data Asset	49
File Attachments: 5 / 5 Risks	50
Note Content: 5 / 5 Risks	51

Session Tokens: 2 / 2 Risks	52
User Credentials: 5 / 5 Risks	53

Trust Boundaries

Application Network	54
Data Tier	54
DMZ	54
Internet	54

Shared Runtime

Docker Host	56
-------------	----

About Threagile

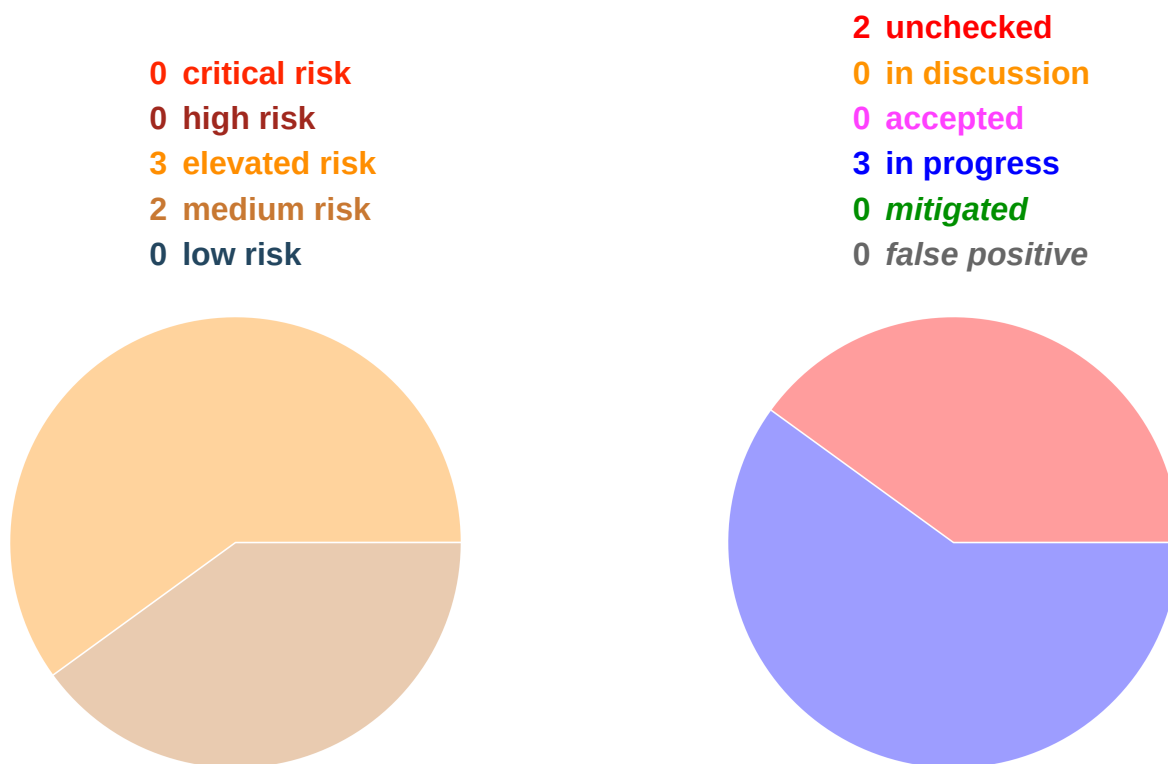
Risk Rules Checked by Threagile	57
Disclaimer	58

Management Summary

Threagile toolkit was used to model the architecture of "VaultNote Threat Model" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "VaultNote Threat Model" whether the mitigation advices have been applied or not.

Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **5 initial risks** in **3 categories** have been identified during the threat modeling process:



VaultNote is a containerized note-taking application comprised of a React SPA, Nginx reverse proxy, Node.js/Express REST API, PostgreSQL database, Redis session store, and MinIO object storage. The architecture deliberately contains several intentional misconfigurations (unencrypted internal HTTP, no encryption at rest, hardcoded credentials) to demonstrate automated threat detection via Threagile. All findings below are expected and serve as demonstration targets.

Impact Analysis of 5 Initial Risks in 3 Categories

The most prevalent impacts of the **5 initial risks** (distributed over **3 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Attackers can enumerate users, brute-force credentials, bulk-download sensitive data, or exhaust backend resources by sending unlimited requests to unthrottled operations. Business process integrity degrades when operations can be repeated without throttling.

Elevated: Stage V - No Rate Limiting on Internet-Exposed API: 1 Initial Risk - Exploitation likelihood is *Likely* with *High* impact.

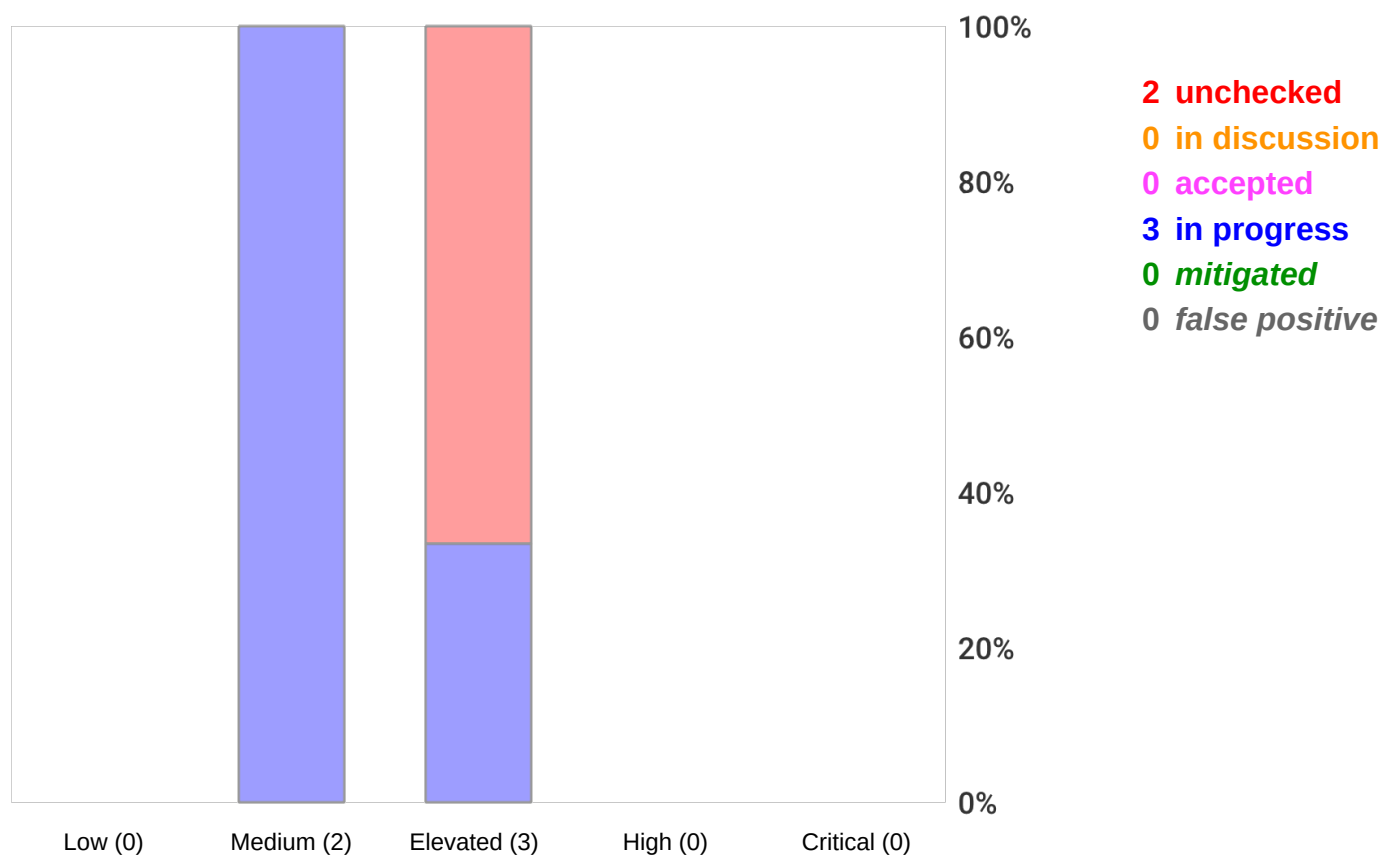
Brute-force attacks against authentication endpoints, mass data harvesting via enumeration, DDoS amplification, and billing abuse on pay-per-use endpoints.

Medium: Stage II - Custom-Developed Asset With No Build Pipeline Security: 2 Initial Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

Injection vulnerabilities, insecure dependencies, hardcoded secrets, and other code-level weaknesses reach production without detection, widening the attack surface.

Risk Mitigation

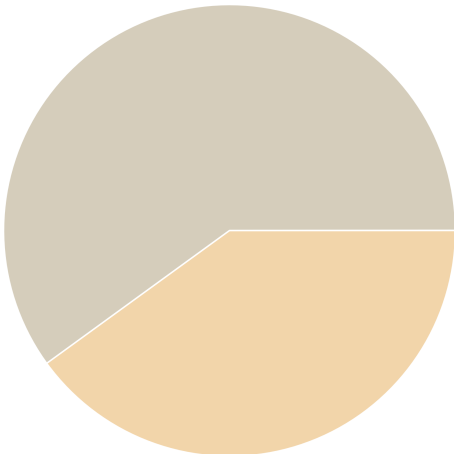
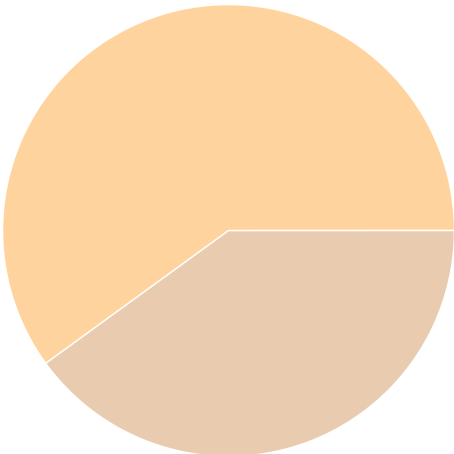
The following chart gives a high-level overview of the risk tracking status (including mitigated risks):



After removal of risks with status *mitigated* and *false positive* the following 5 remain unmitigated:

- 0 unmitigated critical risk
- 0 unmitigated high risk
- 3 unmitigated elevated risk
- 2 unmitigated medium risk
- 0 unmitigated low risk

- 0 business side related
- 0 architecture related
- 2 development related
- 3 operations related



Asset Register

Technical Assets

API Server

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Browser SPA

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

MinIO Object Storage

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

PostgreSQL Database

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Redis Cache

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Data Assets

File Attachments

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

Note Content

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

Session Tokens

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

User Credentials

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

Impact Analysis of 5 Remaining Risks in 3 Categories

The most prevalent impacts of the **5 remaining risks** (distributed over **3 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Attackers can enumerate users, brute-force credentials, bulk-download sensitive data, or exhaust backend resources by sending unlimited requests to unthrottled operations. Business process integrity degrades when operations can be repeated without throttling.

Elevated: Stage V - No Rate Limiting on Internet-Exposed API: 1 Remaining Risk - Exploitation likelihood is *Likely* with *High* impact.

Brute-force attacks against authentication endpoints, mass data harvesting via enumeration, DDoS amplification, and billing abuse on pay-per-use endpoints.

Medium: Stage II - Custom-Developed Asset With No Build Pipeline Security: 2 Remaining Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

Injection vulnerabilities, insecure dependencies, hardcoded secrets, and other code-level weaknesses reach production without detection, widening the attack surface.

Application Overview

Business Criticality

The overall business criticality of "VaultNote Threat Model" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

Business Overview

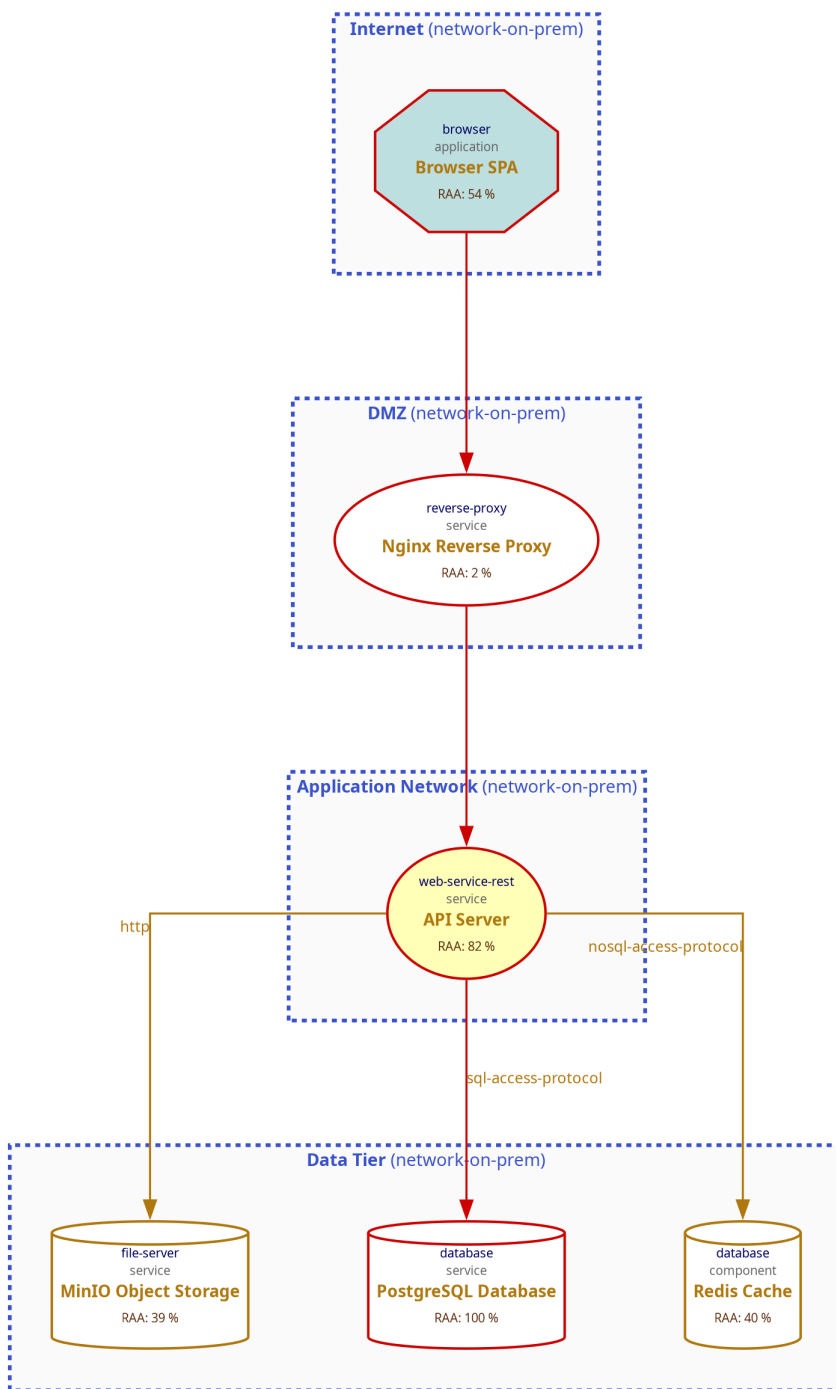
VaultNote allows authenticated users to create, edit, and share encrypted notes with file attachments. The application stores sensitive user data and note content that must be protected against unauthorized disclosure.

Technical Overview

Multi-tier containerized deployment: Nginx terminates TLS and proxies requests to the API over unencrypted HTTP. The API accesses PostgreSQL, Redis, and MinIO over plaintext TCP/HTTP within an internal Docker network.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

cache

Redis Cache

container-runtime

Docker Host

docker

Docker Host

express

API Server

intentional-misconfiguration

MinIO File Storage, MinIO Object Storage, HTTP to API Server

jwt

API Server

minio

MinIO Object Storage

nginx

Nginx Reverse Proxy

nodejs

API Server

object-storage

MinIO Object Storage

postgresql

PostgreSQL Database

react

Browser SPA

redis

Redis Cache

relational

PostgreSQL Database

s3

MinIO Object Storage

session-store

Redis Cache

spa

Browser SPA

tls-termination

Nginx Reverse Proxy

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **5 potential risks** have been identified during the threat modeling process of which **0 in the Spoofing** category, **2 in the Tampering** category, **0 in the Repudiation** category, **0 in the Information Disclosure** category, **3 in the Denial of Service** category, and **0 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

n/a

Tampering

Medium: **Stage II - Custom-Developed Asset With No Build Pipeline Security: 2 / 2 Risks** - Exploitation likelihood is *Unlikely with Medium impact*.

PASTA Stage II defines the technical scope and identifies custom-developed components that require code-level security analysis. A custom-developed technical asset without a build pipeline performing Static Application Security Testing (SAST) means that code vulnerabilities are not caught before deployment, increasing the probability that known vulnerability classes survive to production.

Repudiation

n/a

Information Disclosure

n/a

Denial of Service

Elevated: **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 / 2 Risks** - Exploitation likelihood is *Likely with Medium impact*.

PASTA Stage V identifies vulnerabilities in the attack surface. Custom-developed services that process authentication credentials or session tokens and have outgoing communication links without rate limiting are vulnerable to credential stuffing, brute-force, bulk data exfiltration, and resource exhaustion. Global API rate limits are too coarse to prevent targeted abuse of individual high-value operations such as login, password reset, or bulk data exports.

Elevated: **Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk - Exploitation** likelihood is *Likely* with *High* impact.

PASTA Stage V identifies weaknesses that enable exploitation. An internet-accessible API endpoint without rate limiting is vulnerable to brute-force credential attacks, credential stuffing, API abuse, and denial-of-service amplification. Rate limiting is a fundamental control for any internet-exposed API.

Elevation of Privilege

n/a

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **5 potential risks** have been identified during the threat modeling process of which **0 should be checked by Business Side**, **0 should be checked by Architecture**, **2 should be checked by Development**, and **3 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

n/a

Architecture

n/a

Development

Medium: **Stage II - Custom-Developed Asset With No Build Pipeline Security: 2 / 2 Risks** - Exploitation likelihood is *Unlikely* with *Medium* impact.

Integrate static analysis (e.g., Semgrep, SonarQube, CodeQL) and dependency scanning (e.g., Dependabot, OWASP Dependency-Check) into the build pipeline for every custom-developed asset. Fail builds on high-severity findings.

Operations

Elevated: **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 / 2 Risks** - Exploitation likelihood is *Likely* with *Medium* impact.

Apply rate limiting at the operation level using token-bucket or sliding-window algorithms, enforced per-user for authenticated requests and per-IP for unauthenticated requests. Set `rate_limited`: true on every communication link once per-operation limits are implemented and verified.

Elevated: **Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk** - Exploitation likelihood is *Likely* with *High* impact.

Add rate limiting at the API gateway or application layer (token bucket, sliding window). Apply stricter limits on authentication endpoints. Set `rate_limited` to true on the corresponding communication links after implementation.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

PostgreSQL Database: RAA 100%

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

API Server: RAA 82%

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Browser SPA: RAA 54%

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Redis Cache: RAA 40%

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

MinIO Object Storage: RAA 39%

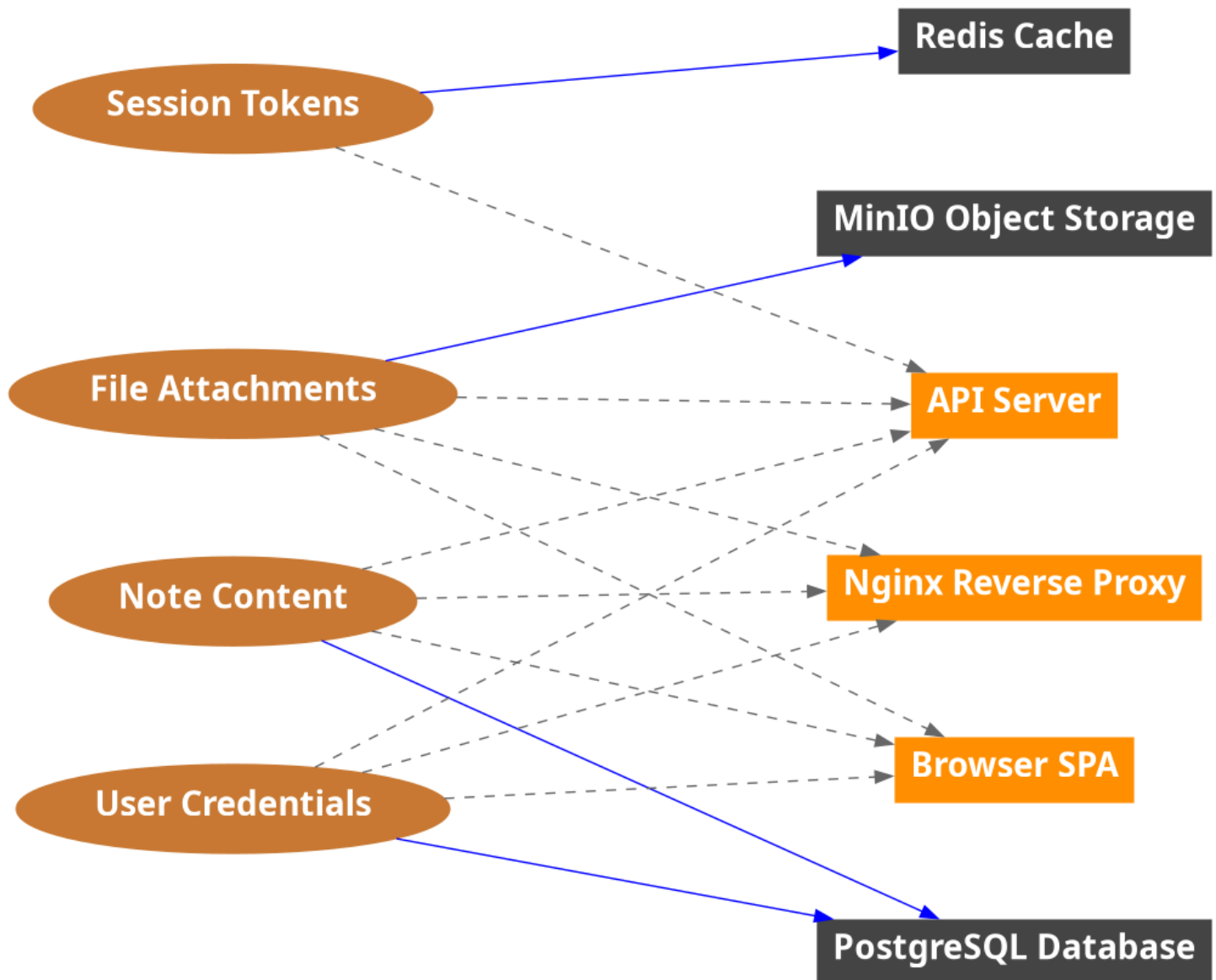
S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy: RAA 2%

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 0 / 0 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

No potential model failures have been identified.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability category

In total **5 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 3 as elevated, 2 as medium, and 0 as low.**

These risks are distributed across **3 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Stage V - Custom-Developed Service Without Per-Operation Rate Limiting:

Description (Denial of Service): [CWE 770](#)

PASTA Stage V identifies vulnerabilities in the attack surface. Custom-developed services that process authentication credentials or session tokens and have outgoing communication links without rate limiting are vulnerable to credential stuffing, brute-force, bulk data exfiltration, and resource exhaustion. Global API rate limits are too coarse to prevent targeted abuse of individual high-value operations such as login, password reset, or bulk data exports.

Impact

Attackers can enumerate users, brute-force credentials, bulk-download sensitive data, or exhaust backend resources by sending unlimited requests to unthrottled operations. Business process integrity degrades when operations can be repeated without throttling.

Detection Logic

In-scope custom-developed process-type technical assets that process authentication data (user-credentials or session-tokens) and have at least one outgoing communication link where `rate_limited` is false and usage is not devops.

Risk Rating

Medium baseline. Elevated for assets with internet-exposed entry points or those processing strictly-confidential authentication credentials.

False Positives

Internal service-to-service links that are not reachable from untrusted callers, or where rate limiting is enforced by an upstream proxy (e.g., Nginx `limit_req`). Set `rate_limited`: true on the link or change usage to devops to suppress.

Mitigation (Operations): Add per-operation rate limiting to all communication links in the authentication and data flow paths

Apply rate limiting at the operation level using token-bucket or sliding-window algorithms, enforced per-user for authenticated requests and per-IP for unauthenticated requests. Set `rate_limited`: true on every communication link once per-operation limits are implemented and verified.

ASVS Chapter: [V4 - Access Control Verification Requirements](#)

Cheat Sheet: [Denial_of_Service_Cheat_Sheet](#)

Check

Is `rate_limited` set to true on every communication link that participates in authentication or sensitive data processing? Are per-user limits enforced in addition to any global API limits?

Risk Findings

The risk **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Per-Operation Rate Limiting: custom service **API Server** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@api-server](#)

Unchecked

Missing Per-Operation Rate Limiting: custom service **Browser SPA** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@browser-spa](#)

Unchecked

Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk

Description (Denial of Service): [CWE 770](#)

PASTA Stage V identifies weaknesses that enable exploitation. An internet-accessible API endpoint without rate limiting is vulnerable to brute-force credential attacks, credential stuffing, API abuse, and denial-of-service amplification. Rate limiting is a fundamental control for any internet-exposed API.

Impact

Brute-force attacks against authentication endpoints, mass data harvesting via enumeration, DDoS amplification, and billing abuse on pay-per-use endpoints.

Detection Logic

In-scope technical assets with entry_point_type of api or web_ui that receive communication links from internet-exposed assets where rate_limited is false.

Risk Rating

Elevated for authentication endpoints; medium for general data APIs. High when the API has no authentication at all.

False Positives

Internal APIs behind a WAF or API gateway that provides rate limiting at the infrastructure layer; document and mark rate_limited as true on the comm links.

Mitigation (Operations): Implement rate limiting on all internet-exposed API entry points

Add rate limiting at the API gateway or application layer (token bucket, sliding window). Apply stricter limits on authentication endpoints. Set rate_limited to true on the corresponding communication links after implementation.

ASVS Chapter: [V4 - Access Control Verification Requirements](#)

Cheat Sheet: [Denial of Service Cheat Sheet](#)

Check

Are all internet-exposed API communication links rate-limited?

Risk Findings

The risk **Stage V - No Rate Limiting on Internet-Exposed API** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Rate Limiting: internet-exposed API **Nginx Reverse Proxy** has communication links without rate limiting: Exploitation likelihood is *Likely* with *High* impact.

[no-rate-limiting-public-api@nginx-proxy](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-120

Rate limiting via nginx limit_req_zone is planned for sprint 21. Current mitigation: account lockout after 10 failed login attempts implemented in the API layer. Full IP-level rate limiting is a VAULT-120 blocker before production launch.

Stage II - Custom-Developed Asset With No Build Pipeline Security: 2 / 2 R

Description (Tampering): [CWE 693](#)

PASTA Stage II defines the technical scope and identifies custom-developed components that require code-level security analysis. A custom-developed technical asset without a build pipeline performing Static Application Security Testing (SAST) means that code vulnerabilities are not caught before deployment, increasing the probability that known vulnerability classes survive to production.

Impact

Injection vulnerabilities, insecure dependencies, hardcoded secrets, and other code-level weaknesses reach production without detection, widening the attack surface.

Detection Logic

In-scope custom-developed technical assets where no sourcecode repository or build-pipeline asset exists in the model to provide build security controls.

Risk Rating

Medium baseline; elevated for internet-exposed custom-developed assets processing sensitive data.

False Positives

Assets where the build pipeline is managed outside this model; document with a reference to the external CI/CD system.

Mitigation (Development): Add SAST and dependency scanning to the CI/CD pipeline

Integrate static analysis (e.g., Semgrep, SonarQube, CodeQL) and dependency scanning (e.g., Dependabot, OWASP Dependency-Check) into the build pipeline for every custom-developed asset. Fail builds on high-severity findings.

ASVS Chapter: [V14 - Configuration Verification Requirements](#)

Cheat Sheet: [Vulnerable_Dependency_Management_Cheat_Sheet](#)

Check

Does every custom-developed technical asset have a build pipeline with SAST and dependency scanning?

Risk Findings

The risk **Stage II - Custom-Developed Asset With No Build Pipeline Security** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

No Build Pipeline Security: custom-developed asset **API Server** has no SAST or dependency scanning: Exploitation likelihood is *Unlikely* with *Medium* impact.

[custom-development-no-sast@api-server](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-121

SAST tooling (Semgrep with Node.js ruleset) is integrated into the GitHub Actions pipeline but the full security rule set is not yet enabled. Remaining rule categories (injection, secrets, SSRF) are being evaluated for false-positive rates. Target: sprint 21.

No Build Pipeline Security: custom-developed asset **Browser SPA** has no SAST or dependency scanning: Exploitation likelihood is *Unlikely* with *Medium* impact.

[custom-development-no-sast@browser-spa](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-121

Same Semgrep pipeline as api-server covers the React SPA source. ESLint security plugin (eslint-plugin-security) is configured but not yet enforced as a blocking check. Planned for sprint 21.

Identified Risks by Technical Asset

In total **5 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 3 as elevated, 2 as medium, and 0 as low.**

These risks are distributed across **6 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

API Server: 2 / 2 Risks

Description

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Per-Operation Rate Limiting: custom service **API Server** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@api-server](#)

Unchecked

Medium Risk Severity

No Build Pipeline Security: custom-developed asset **API Server** has no SAST or dependency scanning: Exploitation likelihood is *Unlikely* with *Medium* impact.

[custom-development-no-sast@api-server](#)

[In Progress](#)

2026-04-25 Security Team

VAULT-121

SAST tooling (Semgrep with Node.js ruleset) is integrated into the GitHub Actions pipeline but the full security rule set is not yet enabled. Remaining rule categories (injection, secrets, SSRF) are being evaluated for false-positive rates. Target: sprint 21.

Asset Information

ID:	api-server
Type:	process
Usage:	business
RAA:	82 %
Size:	service
Technology:	web-service-rest
Tags:	express, jwt, nodejs
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false

Custom-Developed: true
Client by Human: false
Data Processed: File Attachments, Note Content, Session Tokens, User Credentials
Data Stored: none
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: strictly-confidential (rated 5 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: critical (rated 4 in scale of 5)
CIA-Justification: The API processes all user data. It bridges both Docker networks — a compromised API container can pivot into the internal data tier.

Outgoing Communication Links: 3

Target technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (outgoing)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Target: Redis Cache
Protocol: nosql-access-protocol
Encrypted: false
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Sent: Session Tokens
Data Received: Session Tokens

PostgreSQL Connection (outgoing)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Target:	PostgreSQL Database
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Note Content, User Credentials
Data Received:	Note Content, User Credentials

MinIO File Storage (outgoing)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Target:	MinIO Object Storage
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments
Data Received:	File Attachments

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (incoming)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Source:	Nginx Reverse Proxy
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Browser SPA: 2 / 2 Risks

Description

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Per-Operation Rate Limiting: custom service **Browser SPA** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@browser-spa](#)

Unchecked

Medium Risk Severity

No Build Pipeline Security: custom-developed asset **Browser SPA** has no SAST or dependency scanning: Exploitation likelihood is *Unlikely* with *Medium* impact.

[custom-development-no-sast@browser-spa](#)

[In Progress](#)

2026-04-25 Security Team

VAULT-121

Same Semgrep pipeline as api-server covers the React SPA source. ESLint security plugin (eslint-plugin-security) is configured but not yet enforced as a blocking check. Planned for sprint 21.

Asset Information

ID:	browser-spa
Type:	external-entity
Usage:	business
RAA:	54 %
Size:	application
Technology:	browser
Tags:	react, spa
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true

Client by Human:	true
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	End Users
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Client-side code is public; the trust boundary here is user data entered into the browser. XSS would allow an attacker to exfiltrate session tokens.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (outgoing)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Target:	Nginx Reverse Proxy
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Nginx Reverse Proxy: 1 / 1 Risk

Description

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Rate Limiting: internet-exposed API **Nginx Reverse Proxy** has communication links without rate limiting: Exploitation likelihood is *Likely with High* impact.

[no-rate-limiting-public-api@nginx-proxy](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-120

Rate limiting via nginx limit_req_zone is planned for sprint 21. Current mitigation: account lockout after 10 failed login attempts implemented in the API layer. Full IP-level rate limiting is a VAULT-120 blocker before production launch.

Asset Information

ID:	nginx-proxy
Type:	process
Usage:	business
RAA:	2 %
Size:	service
Technology:	reverse-proxy
Tags:	nginx, tls-termination
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Nginx is the single external entry point. Its compromise exposes all downstream services. Availability is critical — outage = full service outage.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (outgoing)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Target:	API Server
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (incoming)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Source:	Browser SPA
Protocol:	https

Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

MinIO Object Storage: 0 / 0 Risks

Description

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	minio-storage
Type:	datastore
Usage:	business
RAA:	39 %
Size:	service
Technology:	file-server
Tags:	intentional-misconfiguration, minio, object-storage, s3
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Stores user file uploads which may contain sensitive documents. Default	

credentials + no encryption = trivial exfiltration by any process on backend-net.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

MinIO File Storage (incoming)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Source:	API Server
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments
Data Sent:	File Attachments

PostgreSQL Database: 0 / 0 Risks

Description

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	postgresql-db
Type:	datastore
Usage:	business
RAA:	100 %
Size:	service
Technology:	database
Tags:	postgresql, relational
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Holds all user PII (email, hashed passwords) and all note content. Strictly	

confidential — unencrypted disk access exposes all data.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

PostgreSQL Connection (incoming)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Source:	API Server
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Note Content, User Credentials
Data Sent:	Note Content, User Credentials

Redis Cache: 0 / 0 Risks

Description

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	redis-cache
Type:	datastore
Usage:	business
RAA:	40 %
Size:	component
Technology:	database
Tags:	cache, redis, session-store
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Session Tokens
Data Stored:	Session Tokens
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Session token exposure enables account takeover. Non-persistent by	

default but the container volume could persist RDB snapshots.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (incoming)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Source:	API Server
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Session Tokens
Data Sent:	Session Tokens

Identified Data Breach Probabilities by Data Asset

In total **5 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 3 as elevated, 2 as medium, and 0 as low.**

These risks are distributed across **4 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

File Attachments: 5 / 5 Risks

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

ID:	file-attachments		
Usage:	business		
Quantity:	many		
Tags:	none		
Origin:	customer		
Owner:	VaultNote Team		
Confidentiality:	confidential	(rated 4 in scale of 5)	
Integrity:	critical	(rated 4 in scale of 5)	
Availability:	operational	(rated 2 in scale of 5)	
CIA-Justification:	Files may contain sensitive documents. Integrity is critical because tampered attachments corrupt evidence or business records. MinIO's default credentials (minioadmin) make all stored files accessible to anyone on the network — intentional misconfiguration for demo purposes.		
Processed by:	API Server, Browser SPA, MinIO Object Storage, Nginx Reverse Proxy		
Stored by:	MinIO Object Storage		
Sent via:	MinIO File Storage, HTTPS to Nginx, HTTP to API Server		
Received via:	MinIO File Storage, HTTPS to Nginx, HTTP to API Server		
Data Breach:	possible		
Data Breach Risks:	This data asset has data breach potential because of 5 remaining risksStr:		
	Possible: per-operation-rate-limiting-missing@api-server		
	Possible: per-operation-rate-limiting-missing@browser-spa		
	Possible: custom-development-no-sast@api-server		
	Possible: custom-development-no-sast@browser-spa		
	Possible: no-rate-limiting-public-api@nginx-proxy		

Note Content: 5 / 5 Risks

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

ID:	note-content	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Notes may contain highly sensitive personal information. Integrity is critical because corrupted notes lose user trust.	
Processed by:	API Server, Browser SPA, Nginx Reverse Proxy, PostgreSQL Database	
Stored by:	PostgreSQL Database	
Sent via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Received via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 5 remaining risksStr:	
	Possible: per-operation-rate-limiting-missing@api-server	
	Possible: per-operation-rate-limiting-missing@browser-spa	
	Possible: custom-development-no-sast@api-server	
	Possible: custom-development-no-sast@browser-spa	
	Possible: no-rate-limiting-public-api@nginx-proxy	

Session Tokens: 2 / 2 Risks

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

ID:	session-tokens	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Session token compromise enables full session hijacking. Short TTLs and revocation lists mitigate but do not eliminate the risk.	
Processed by:	API Server, Redis Cache	
Stored by:	Redis Cache	
Sent via:	Redis Session Store	
Received via:	Redis Session Store	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 2 remaining risksStr:	
	Possible: per-operation-rate-limiting-missing@api-server	
	Possible: custom-development-no-sast@api-server	

User Credentials: 5 / 5 Risks

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

ID:	user-credentials
Usage:	business
Quantity:	many
Tags:	none
Origin:	customer
Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Credentials grant full account access. A breach leaks all users' identities and enables account takeover across every note and attachment.
Processed by:	API Server, Browser SPA, Nginx Reverse Proxy, PostgreSQL Database
Stored by:	PostgreSQL Database
Sent via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server
Received via:	PostgreSQL Connection

Data Breach: **possible**

Data Breach Risks: This data asset has data breach potential because of 5 remaining risksStr:

Possible: per-operation-rate-limiting-missing@api-server

Possible: per-operation-rate-limiting-missing@browser-spa

Possible: custom-development-no-sast@api-server

Possible: custom-development-no-sast@browser-spa

Possible: no-rate-limiting-public-api@nginx-proxy

Trust Boundaries

In total **4 trust boundaries** have been modeled during the threat modeling process.

Application Network

Internal Docker bridge network (frontend-net). Contains the API server which is reachable from Nginx but also bridges into backend-net.

ID: application-network
Type: network-on-prem
Tags: none
Assets inside: API Server
Boundaries nested: none

DMZ

Demilitarized zone containing the Nginx reverse proxy. Accessible from the internet but isolated from the application and data tiers.

ID: dmz
Type: network-on-prem
Tags: none
Assets inside: Nginx Reverse Proxy
Boundaries nested: none

Data Tier

Internal Docker bridge network (backend-net, marked internal: true). No external routing. Contains all datastores. The API server has a NIC on this network and acts as the sole legitimate access path.

ID: data-tier
Type: network-on-prem
Tags: none
Assets inside: MinIO Object Storage, PostgreSQL Database, Redis Cache
Boundaries nested: none

Internet

Untrusted external network. All traffic from this zone must cross the Nginx TLS termination point before entering the DMZ.

ID: internet
Type: [network-on-prem](#)
Tags: none
Assets inside: Browser SPA
Boundaries nested: none

Shared Runtimes

In total **1 shared runtime** has been modeled during the threat modeling process.

Docker Host

All containers run on the same Docker host. A container escape or privileged container could access the host filesystem including Docker socket and database volumes.

ID:	docker-host
Tags:	container-runtime, docker
Assets running:	Nginx Reverse Proxy, API Server, PostgreSQL Database, Redis Cache, MinIO Object Storage

Risk Rules Checked by Threagile

Threagile Version: 1.0.0

Threagile Build Timestamp:

Threagile Execution Timestamp: 20260524151525

Model Filename: /home/coder221/Desktop/coding/Threat-model/threagile/threagile.yaml

Model Hash (SHA256): d7bfa142abb73e5d3cd46eac265cb228675b53c92a8d976eddf1abb34b52c1d1

Threagile (see <https://threagile.io> for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (<https://christian-schneider.net>): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Disclaimer

VaultNote Security Team conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. VaultNote Security Team and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that VaultNote Security Team and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a time box approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.